

1. Write a procedure that computes the perimeter and the area of a rectangle. Define your own values for the length and width. (Assuming that L and W are the length and width of the rectangle, Perimeter = $2*(L+W)$ and Area = $L*W$).

```
KD3-99107-shashank@localhost> Create procedure rect_peri_sp( l double, w double)
-> BEGIN
-> DECLARE v_area1 double;
-> DECLARE v_peri double;
-> SET v_area1 = 2*(l * w);
-> SET V_peri = l*w;
-> insert into que1 values(0.0, v_area1, V_peri);
-> END;
-> $$
Query OK, 0 rows affected (0.005 sec)

KD3-99107-shashank@localhost>
KD3-99107-shashank@localhost> DELIMITER ;
KD3-99107-shashank@localhost> call rect_peri_sp(2,3);
Query OK, 1 row affected (0.006 sec)

KD3-99107-shashank@localhost> select * from que1;
+-----+-----+-----+
| value | peri | area1 |
+-----+-----+-----+
|      0 |    12 |      6 |
+-----+-----+-----+
1 row in set (0.001 sec)

KD3-99107-shashank@localhost> 
```

Bin

2. Write a **procedure** that declares an **integer** variable called num, assigns a **value** to it, computes and inserts into the temp table the **value** of the variable itself, its square, and its **cube**.

```
KD3-99107-shashank@localhost> DELIMITER $$
KD3-99107-shashank@localhost>
KD3-99107-shashank@localhost> Create procedure value_square_cube( num double)
-> BEGIN
-> DECLARE V_sqr double;
-> DECLARE V_cb double;
-> SET V_sqr = num * num;
-> SET V_cb = num * num * num;
-> INSERT INTO temp values(num, V_sqr, V_cb);
-> END;
-> $$
Query OK, 0 rows affected (0.004 sec)

KD3-99107-shashank@localhost>
KD3-99107-shashank@localhost> DELIMITER ;
KD3-99107-shashank@localhost> call value_square_cube(3);
Query OK, 1 row affected (0.005 sec)

KD3-99107-shashank@localhost> select * from temp;
+-----+-----+-----+
| value | sqr  | cb   |
+-----+-----+-----+
|      3 |     9 |    27 |
+-----+-----+-----+
1 row in set (0.001 sec)

KD3-99107-shashank@localhost> call value_square_cube(5);
Query OK, 1 row affected (0.002 sec)

KD3-99107-shashank@localhost> select * from temp;
+-----+-----+-----+
| value | sqr  | cb   |
+-----+-----+-----+
|      3 |     9 |    27 |
|      5 |    25 |   125 |
+-----+-----+-----+
2 rows in set (0.001 sec)

KD3-99107-shashank@localhost> 
```

Create a **procedure** to Convert a temperature in Fahrenheit (F) to its equivalent in Celsius (C) and vice versa. The **required** formulae are

$$C = (F - 32) * 5/9 \quad F = 9/5 * C + 32$$

```

^C
KD3-99107-shashank@localhost> Delimiter $$
KD3-99107-shashank@localhost> Create procedure ConvertTemp(IN temp_value double, IN conv_type varchar(5), OUT result double)
--> BEGIN
--> If conv_type = 'F2C' then
--> Set result = (temp_value - 32) * 5/9;
--> ElseIf conv_type = 'C2F' then
--> Set result = (9/5 * temp_value) + 32;
--> Else
--> Set result = null;
--> End if;
--> End;
--> $$
Query OK, 0 rows affected (0.068 sec)

KD3-99107-shashank@localhost> Delimiter ;
KD3-99107-shashank@localhost> CALL ConvertTemp(98.6, 'F2C', @res);
Query OK, 0 rows affected (0.006 sec)

KD3-99107-shashank@localhost> SELECT @res;
+-----+
| @res |
+-----+
|    37 |
+-----+
1 row in set (0.002 sec)

KD3-99107-shashank@localhost>
KD3-99107-shashank@localhost> CALL ConvertTemp(37, 'C2F', @res);
Query OK, 0 rows affected (0.000 sec)

KD3-99107-shashank@localhost> SELECT @res;
+-----+
| @res |
+-----+
| 98.60000000000001 |
+-----+
1 row in set (0.000 sec)

KD3-99107-shashank@localhost>

```

4. Create a procedure to convert a number of inches into yards, feet, and inches. For example, 124 inches equals 3 yards, 1 foot, and 4 inches.

```

KD3-99107-shashank@localhost> Delimiter $$
KD3-99107-shashank@localhost> Create procedure ConvertInches(IN total_inches int, OUT yards int, OUT
feet int, OUT inches int)
    -> BEGIN
    -> Set yards = total_inches DIV 36;
    -> Set total_inches = total_inches MOD 36;
    -> Set feet = total_inches DIV 12;
    -> Set inches = total_inches MOD 12;
    -> End;
    -> $$
Query OK, 0 rows affected (0.007 sec)

KD3-99107-shashank@localhost> Delimiter ;
KD3-99107-shashank@localhost>
KD3-99107-shashank@localhost> CALL ConvertInches(124, @y, @f, @i);
Query OK, 0 rows affected (0.003 sec)

KD3-99107-shashank@localhost> SELECT @y as yards, @f as feet, @i as inches;
+-----+-----+-----+
| yards | feet | inches |
+-----+-----+-----+
|      3 |     1 |       4 |
+-----+-----+-----+
1 row in set (0.002 sec)

```

5. Create a procedure to read two real numbers and tell whether the product of the two numbers is equal to or greater than 100.

```

KD3-99107-shashank@localhost> Delimiter $$
KD3-99107-shashank@localhost> Create procedure CheckProduct(IN num1 double, IN num2 double, OUT resu
lt varchar(30))
-> BEGIN
-> declare prod double;
-> Set prod = num1 * num2;
-> If prod >= 100 then
-> Set result = 'Greater than or equal to 100';
-> Else
-> Set result = 'Less than 100';
-> End if;
-> End;
-> $$
Query OK, 0 rows affected (0.004 sec)

KD3-99107-shashank@localhost> Delimiter ;
KD3-99107-shashank@localhost>
KD3-99107-shashank@localhost> CALL CheckProduct(10, 15, @res);
Query OK, 0 rows affected (0.002 sec)

KD3-99107-shashank@localhost> SELECT @res;
+-----+
| @res          |
+-----+
| Greater than or equal to 100 |
+-----+
1 row in set (0.001 sec)

```